

Batch Reprint and Print Automation

Last Updated: 2026-10-02

Companion to docs/specs/BatchSkidEdit_UX_Specification.md . That document ends with correctly retagged skids. This document covers what happens next: getting a new physical label onto each of those skids, handing the batch to Module_Reprint without re-searching for it, and printing a report of what changed.

What Already Exists

The reprint capability was built recently and is substantially correct. This spec extends it rather than replacing it.

Piece	Location	Behavior today
Reprint landing page	Module_Reprint/Views/View_Reprint_Main.xaml	Three mode cards: Receiving, Dunnage, Volvo
History page	Module_Reprint/Views/View_Reprint_ModulePage.xaml	Filters row, 5-column history list, footer with Select All and Reprint
Shared page behavior	ViewModel_Reprint_ModuleBase	Date range plus presets, search-by, column choice, select-all, ReprintAsync , ReprintCompleted event
Per-module queueing	Service_Reprint_Receiving , Service_Reprint_Dunnage , Service_Reprint_Volvo	GetReprintHistoryAsync and ReprintAsync(historyIds)
Result bucketing	Module_Core/Models/Reprint/Model_ReprintBatchResult	Queued , AlreadyQueued , Failed
History row shape	Module_Core/Models/Reprint/Model_ReprintHistoryRow	HistoryId , RecordDate , Part , Quantity , Reference , AlreadyQueued
Physical label launch	Module_Core/Services/Service_LabelViewLauncher via IService_LabelViewLauncher.LaunchLabelAsync(labelFilePath)	Validates the .lb1 path, resolves the LabelView executable, opens the file
Already-queued protection	View_Reprint_ModulePage.xaml red row background plus CanSelect	Model_ReprintBatchResult.AlreadyQueued reported in the completion summary

Reprint semantics: a selected history row is **re-queued** into the active label queue with is_reprint = 1 . LabelView then reads that queue through the .lb1 data source when the label is

launched. Queuing and printing are therefore two separate physical steps, and that is the source of most of the friction below.

The Gap

The attached request exposes five problems in the current handoff.

#	Problem	Consequence for the operator
1	Reprint has no concept of a batch	After retagging three skids, the operator must re-find them by date range and part number search
2	Default date range excludes recent corrections	A batch retagged yesterday is invisible unless the operator widens the range
3	The history grid shows date, part, quantity, reference only	The operator cannot verify Location or Heat/Lot on the row, which is exactly what the transfer request is about
4	"Already queued" is red and unselectable but unexplained	The operator cannot tell whether the row was queued by today's batch or a previous day's
5	Queuing does not launch the label	The operator must leave the app, find the label file, and print from LabelView manually

Problem 5 is why "Print Entire Batch" must be part of this design rather than a separate feature.

The Handoff Contract

`Module_Receiving` must not reach into `Module_Reprint` internals, and `Module_Reprint` must not query `Receiving` history to reconstruct what happened. Both already depend on `Module_Core`, so the shared contract belongs there.

New model

`Module_Core/Models/Reprint/Model_ReprintBatchHandoff.cs`

Property	Type	Purpose
BatchId	string	Human-readable id, for example B-20261002-001 . Shown in the Reprint UI.
SourceModule	Enum_ReprintMode	Restricted to Receiving for this workflow
SourcePartId	string	The part the skids were, for example MMC000659
TargetPartId	string	The part the skids now are, for example MMC000767
AffectedHistoryIds	IReadOnlyList<string>	HistoryId values matching Model_ReprintHistoryRow.HistoryId
AffectedLocations	IReadOnlyList<string>	Location ids of the affected skids, for verification in the Reprint grid

Property	Type	Purpose
SelectedTotal	decimal	Sum of the affected skids' quantities
ExpectedTotal	decimal?	What the transfer request said it should be, if entered
Variance	decimal?	SelectedTotal - ExpectedTotal
CreatedUtc	DateTime	When the batch was applied
CreatedByUserId	string?	Operator identity

New service

Module_Core/Contracts/Services/IService_ReprintBatchHandoff.cs plus a session-scoped implementation alongside the existing Module_Core services, registered with the other services in Infrastructure/DependencyInjection/ .

Member	Behavior
void Set(Model_ReprintBatchHandoff handoff)	Called by ViewModel_Receiving_EditMode after a successful save
Model_ReprintBatchHandoff? Current { get; }	Read by ViewModel_Reprint_ModuleBase when the page activates
event EventHandler? HandoffChanged	Lets the Reprint page refresh if the batch is replaced while open
void Clear()	Called by the Reprint page's "Clear batch filter" action

Session-scoped, in-memory, no schema change. If the app restarts, the batch filter is gone and the operator falls back to date range search — the same as today, so nothing regresses.

Handoff sequence

Receiving Edit Mode

1. Operator selects 3 skids, runs Batch Edit, Part ID -> MMC000767
2. Operator presses Save & Finish -> SaveAsync persists the rows
3. On success only, view model builds Model_ReprintBatchHandoff from the saved rows' negative/positive ids + locations + totals
4. IService_ReprintBatchHandoff.Set(handoff)
5. Selection Bar surfaces a "Reprint Labels" action, badged with the batch id

Module_Reprint

6. Page activates, reads Current, applies the batch filter automatically
7. Grid shows exactly the 3 affected skids, already checked
8. Operator presses "Print Batch (3)"

Step 3 must be gated on save success. Handing off a batch that failed to persist would produce labels for a change that does not exist in the database.

Reprint Filter Surface

Additions to the existing filters row in `View_Reprint_ModulePage.xaml` . The row is already at eight columns, so the new controls go in a second chip row directly beneath it rather than forcing the existing controls to shrink.

Batch and recency chips

Chip	Behavior
Current batch (3)	Filters to <code>AffectedHistoryIds</code> . Only present when a handoff exists. Selected by default after a handoff. Shows a tooltip with <code>SourcePartId</code> → <code>TargetPartId</code> and the operator.
Recently modified · 24h	Filters to rows whose history record was written in the last 24 hours. This is the recovery path when there is no handoff, and it is what makes a retag findable the next morning.
Recently modified · 7d	Same, seven-day window
Changed to <part>	Appears when a handoff exists; filters to <code>TargetPartId</code>
Clear batch filter	Explicit exit from the batch context back to normal date-range search

New search-by option

`BuildSearchByOptions()` in `ViewModel_Reprint_Receiving` gains a `Location` option and a `Heat/Lot` option. Part is already present. This matters because "which skid was in V-F0-07" is the natural question after a location-based transfer request.

New columns

`Model_ReprintColumnOption` already drives the column chooser in `Dialog_Reprint_ColumnChooser.xaml` . Add `Location` and `Heat/Lot` options, and extend the header row and row template in `View_Reprint_ModulePage.xaml` accordingly. `Model_ReprintHistoryRow` gains `Location` and `HeatLot` string properties, populated by the per-module history queries.

Default visibility: `Location` visible, `Heat/Lot` hidden. The header row must stay in sync with whatever the row template renders, since both are hand-authored grids rather than a `DataGrid` with auto-generated columns.

Batch context banner

When a handoff is active, an `InfoBar` sits above the history list, styled consistently with the existing "Already queued" `InfoBar`:

Batch

B-20261002-001 — 3 skids retagged from MMC000659 to MMC000767 by jkoll on 10/2/2026. Selected total 236,634 lb against expected 154,540 lb (variance +82,094 lb).

The variance sentence only appears when `ExpectedTotal` was entered. This is the single place where the physical label run is tied back to the arithmetic in the transfer request.

Already-queued messaging

Replace the generic red-row treatment with a specific reason, sourced from what `Model_ReprintBatchResult.AlreadyQueued` already tells the view model:

Row state	Row treatment	Tooltip
Queued by the current batch	Green tint, checkbox hidden	"Already queued for this batch"
Queued by an earlier run	Red tint, checkbox hidden	"Queued on 10/1/2026 4:12 PM by mlaurin"
Not queued	Normal, checkbox visible	—

Single-Click Print Batch

The paradigm: one button, one confirmation, one progress surface, one result.

Interaction

- Footer primary button reads `Print Batch (3) — ReprintButtonLabel` extended to include the count when a handoff is active, falling back to its current wording otherwise.
- Pressing it opens a confirmation dialog, because physical labels are hard to un-print:

```
Print 3 replacement labels?  
MMC000659 -> MMC000767  
V-B0-00 · V-F0-03 · V-F0-07  
Total 236,634 lb      Expected 154,540 lb      Variance +82,094 lb  [does not balance]  
A "does not balance" note will be printed on the batch report.  
[ Print 3 labels ]   [ Cancel ]
```

- On confirm, the existing queueing path runs: `ExecuteReprintAsync(AffectedHistoryIds)` through `IService_Reprint_Receiving.ReprintAsync`.
- On queue success, the physical step runs: `IService_LabelViewLauncher.LaunchLabelAsync(labelPath)` using the label file already configured for the receiving label station. No second launch mechanism is introduced.
- Result summary reuses the existing `ReprintCompleted` handling in `View_Reprint_ModulePage.xaml.cs`, extended with a `Print again` action for the failed bucket.

Outcome handling

Queue outcome	Operator sees	Available action
All queued, label launched	"3 labels queued. Label opened in LabelView."	View batch report
Some already queued	"2 queued, 1 was already queued. Label opened in LabelView."	View batch report
All already queued	Amber: "All 3 skids are already queued. Nothing new was added."	Reprint anyway (re-queues) or Cancel

Queue outcome	Operator sees	Available action
Some failed	Red summary listing the failed references, checkbox state preserved on failed rows	Retry failed (1)
Label launch failed	Queued successfully, launch failed, with the validation message from <code>Service_LabelViewLauncher</code>	Open label manually , Retry

The distinction between "queued" and "printed" must be visible in the copy. Claiming "printed" when LabelView merely opened would be a lie the operator will eventually catch.

Progress and responsiveness

Queueing N rows is per-row `Model_Dao_Result` work. The progress surface shows `Queueing 12 of 40` and the window stays responsive; see `Module_Shared/Proposed_Validation_Safeguards.md` for the threading contract.

Batch Report Printing

The operator needs a paper artifact for the transfer package. `Module_Reporting` already builds a printable document shape but has **no print or export path at all** today — `Service_Reporting` exposes `GetReceivingHistoryAsync` , `BuildSummaryTablesAsync` , and `FormatForEmailAsync` , and the only output destination is the clipboard via `Service_ReportingClipboard` . So this is genuinely new work, and it is scoped to reuse the existing document model rather than invent a second one.

Design

Step	Mechanism
Gather	Selected rows plus the handoff metadata, in <code>ViewModel_Receiving_EditMode</code> or the Reprint page
Shape	<code>Model_FormattedReportDocument</code> with two summary tables: <code>Skids changed</code> and <code>Reconciliation</code>
Render	New print surface that lays the formatted document out for a page, not for email
Output	System print dialog, plus <code>Save as PDF</code> and <code>Copy to clipboard</code>

Report contents

Section	Rows
Header	Batch id, applied timestamp, operator, source mode (<code>Memory</code> / <code>Current Labels</code> / <code>History</code>)
Transfer	<code>MMC000659</code> → <code>MMC000767</code> , the requesting purchase order references, the quoted expected total
Table 1 — Skids changed	Location, Heat/Lot, quantity, before part, after part

Section	Rows
Table 2 — Reconciliation	Selected total, expected total, variance, balance status, and an explicit statement when out of tolerance
Footer	Signature line for the person who physically re-labeled and verified the skids

That signature line is deliberate: the handwritten note says *"I will fix the system when completed"*, which means a human is reconciling afterward. Giving that human a signed record is the difference between a correction and a mystery.

Required Tweaks to Existing Reprint Code

Ordered smallest-first so the low-risk wins land early.

1. `Model_ReprintHistoryRow` — add `Location` and `HeatLot` .
2. `Model_ReprintColumnOption` plus `Dialog_Reprint_ColumnChooser` — add the two column options.
3. `ViewModel_Reprint_ModuleBase.BuildSearchByOptions` overrides — add `Location` , `Heat/Lot` .
4. `View_Reprint_ModulePage.xaml` — header row and row template gain the two columns; add the chip row and the batch context `InfoBar` .
5. `ViewModel_Reprint_ModuleBase` — add batch filter state, the `Current` handoff read on activate, and the recency filter. `ActivateAsync` currently defaults to today, which is the behavior that hides yesterday's correction.
6. `ViewModel_Reprint_Receiving` — map `Location` and `HeatLot` in the history projection.
7. `View_Reprint_ModulePage.xaml.cs` — `OnReprintCompleted` gains the print follow-on and the `Retry failed` path.
8. `Infrastructure/DependencyInjection/` — register `IService_ReprintBatchHandoff` .

Out of Scope

- No change to the `is_reprint = 1` queue semantics.
- No new stored procedures. Reprint already has what it needs.
- No `LabelView .lb1` file changes. `Service_LabelViewLauncher` behavior is untouched.
- No email or unattended printing. `FormatForEmailAsync` is not repurposed here.
- No automatic print triggered by save. Physical output always requires a click and a confirmation.

Acceptance Criteria

- ☐ After a batch retag saves, navigating to Reprint → Receiving shows the batch chip and the affected skids already filtered and checked, with no manual searching.
- ☐ A batch retagged yesterday is findable through the `Recently modified` chips.
- ☐ `Location` and `Heat/Lot` are available as columns and as search-by options.
- ☐ Already-queued rows state who queued them and when.
- ☐ `Print Batch (N)` queues the batch and opens the label in one interaction, with a confirmation that shows the reconciliation state.

- ☐ The result summary distinguishes queued from printed, and offers `Retry` `failed` when a partial failure occurs.
- ☐ A batch report can be printed and saved as PDF, containing both the change table and the reconciliation section.
- ☐ All new user-facing strings come from settings keys, consistent with the existing `Reprint` `ReprintSettingsKeys` and `Receiving` `ReceivingSettingsKeys` patterns.